

Advanced CNNs & Transfer Learning

AI Accelerator Program

Session 22 | 4:30 PM – 6:30

Instructor;
Mahi Aminu Aliyu
AI & SI Associate AHFID



Opening Story: The Impossible Transfer 🤯

Imagine this experiment:

1. Train a CNN to recognize **dog breeds** (Labrador, Chihuahua, German Shepherd...)
2. Take that same CNN
3. Use it to recognize **handwritten digits** (0, 1, 2, 3...)

Question: Does this make ANY sense? Dogs and numbers look NOTHING alike!

The Surprising Result

It works. And it works **BETTER** than training from scratch!

Researchers discovered:

- The CNN trained on dogs learned **universal visual patterns**
- Edge detectors for dog fur → Perfect for digit strokes ✓
- Curve detectors for dog ears → Perfect for "3" and "8" ✓
- Texture patterns for dog eyes → Perfect for distinguishing "6" from "9" ✓

This is transfer learning: Models learn representations that transfer across completely different domains!

Why This Matters for You

You don't need to train CNNs from scratch on millions of images.

Instead:

- Start with a model trained on ImageNet (1.4M images, 1000 classes)
- Transfer that knowledge to **YOUR** problem (Nigerian crop disease, road conditions, etc.)
- Train in **minutes** instead of days
- Get **better accuracy** with **less data**

Today: Learn how to leverage models trained by Google, Facebook, Microsoft!

Learning Objectives

By the end of today, you will:

1. ✓ Understand modern CNN architectures (ResNet, EfficientNet)
2. ✓ Apply transfer learning (feature extraction & fine-tuning)
3. ✓ Use data augmentation to improve performance
4. ✓ Implement optimization tricks (early stopping, LR scheduling)
5. ✓ Build a Nigerian crop disease classifier with transfer learning

Today's Nigerian Use Case

Crop Disease Detection for Nigerian Farmers

Problem:

- Cassava, maize, and yam diseases devastate harvests
- Farmers lack access to agricultural extension officers
- Early detection = 70% yield loss prevented

Solution:

- Mobile app with CNN classifier
- Take photo of leaf → Instant diagnosis
- Transfer learning makes this feasible with limited labeled data

The Dataset: Nigerian Crop Diseases

Classes:

- Cassava: Mosaic disease, Brown streak, Healthy
- Maize: Northern leaf blight, Common rust, Healthy
- Yam: Anthracnose, Healthy

Challenge:

- Only ~500 images per class (small dataset!)
- Variable lighting conditions (farm fields)
- Different camera qualities (smartphone diversity)

Transfer learning solves all three problems!



Part 1: Modern CNN Architectures



Evolution of CNNs

Model	Year	Key Innovation	ImageNet Top-5 Error
AlexNet	2012	Deep learning breakout	15.3%
VGG16	2014	Depth matters (16 layers)	7.3%
ResNet50	2015	Skip connections	3.6%
EfficientNet	2019	Compound scaling	1.9%

Today's focus: ResNet (skip connections) + EfficientNet (efficient scaling)

The Vanishing Gradient Problem

Traditional deep networks (50+ layers):

- Gradients get smaller as they backpropagate
- Early layers stop learning
- Network performs WORSE as you add layers (counterintuitive!)

Example:

- 20-layer network: 90% accuracy ✓
- 50-layer network: 85% accuracy ✗

Why? Information and gradients get lost in deep stacks.

The Vanishing Gradient Problem

Traditional deep networks (50+ layers):

- Gradients get smaller as they backpropagate
- Early layers stop learning
- Network performs WORSE as you add layers (counterintuitive!)

Example:

- 20-layer network: 90% accuracy ✓
- 50-layer network: 85% accuracy ✗

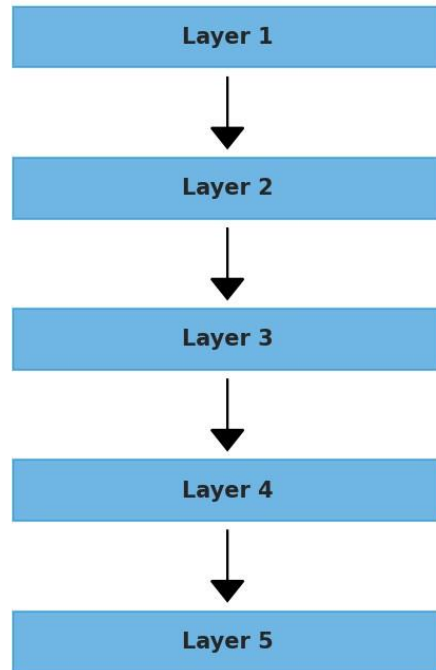
Why? Information and gradients get lost in deep stacks.

ResNet: Skip Connections to the Rescue

ResNet Architecture: Skip Connections Solve Vanishing Gradients

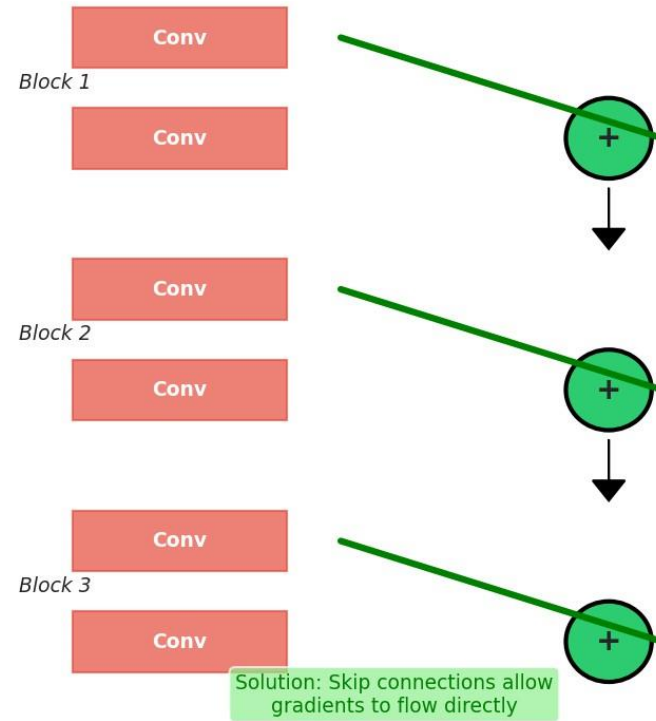
— Skip

Traditional Deep Network



Problem: Vanishing gradients
in very deep networks

ResNet: Residual Learning



How Skip Connections Work

Traditional block:

Input $x \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Output}$

Residual block:

Input x $\xrightarrow{\hspace{10em}}$ $+$ $\rightarrow$ Output
 $\downarrow$
 $\rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{ReLU} \xrightarrow{\hspace{1em}}$
 $\uparrow$

Benefits:

1. Gradients flow directly through skip connections
2. Network can learn identity mapping if needed
3. Enables training 100+ layer networks

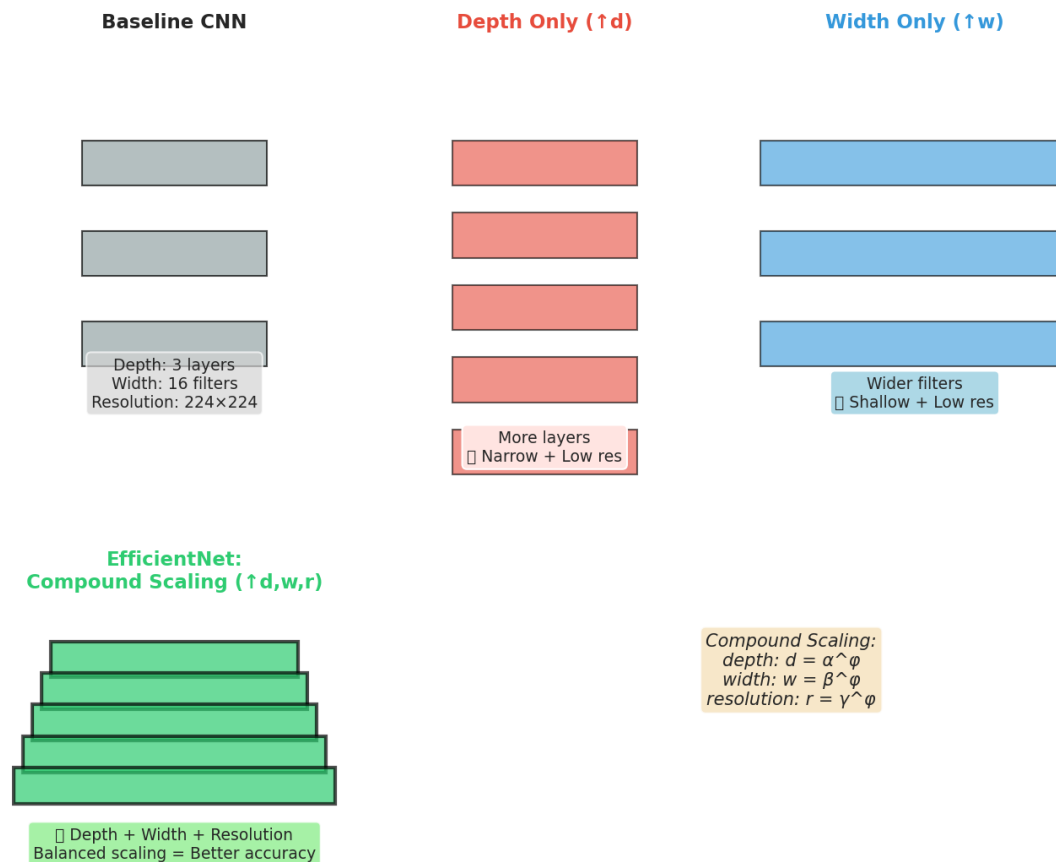
ResNet Variants

Model	Layers	Parameters	Use Case
ResNet50	50	25M	General purpose (balanced)
ResNet101	101	44M	Higher accuracy needed
ResNet152	152	60M	State-of-the-art (slow)

For Nigerian crop disease: ResNet50 is the sweet spot 

EfficientNet: Compound Scaling

EfficientNet: Compound Scaling Beats Single-Dimension Scaling



Compound Scaling Formula

Baseline model: $d=1$, $w=1$, $r=224 \times 224$

Scale with coefficient φ :

- **Depth:** $d = \alpha^{\varphi}$
- **Width:** $w = \beta^{\varphi}$
- **Resolution:** $r = \gamma^{\varphi}$

Constraint: $\alpha \times \beta^2 \times \gamma^2 \approx 2$

Result: EfficientNetB0 $\rightarrow$ B7 with systematic scaling

EfficientNet Variants

Model	Params	ImageNet Acc	Speed
EfficientNetB0	5.3M	77.1%	Fast
EfficientNetB3	12M	81.6%	Medium
EfficientNetB7	66M	84.3%	Slow

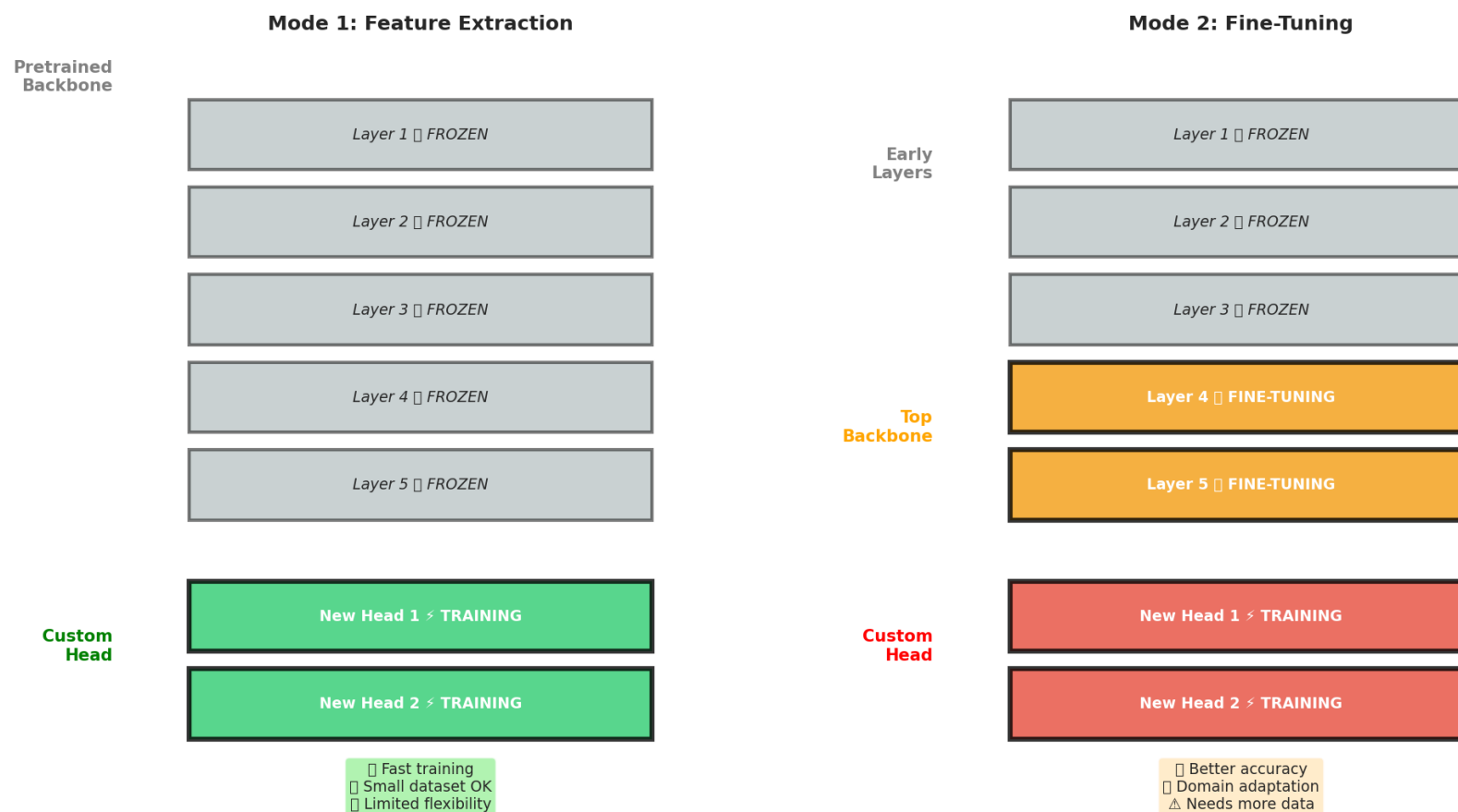
Why it matters: B0 achieves ResNet50 accuracy with 5× fewer parameters!

For mobile deployment in Nigeria: EfficientNetB0 is ideal 📱

Part 2: Transfer Learning Strategies

Two Transfer Learning Modes

Two Transfer Learning Strategies



Mode 1: Feature Extraction

When to use:

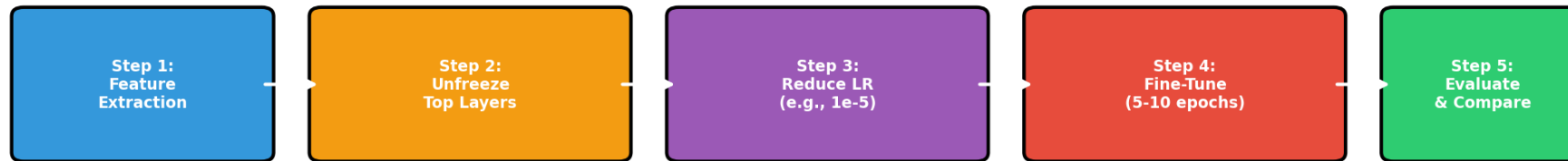
- Small dataset (<5,000 images)
- New task similar to ImageNet (e.g., other objects/animals)
- Limited compute resources
- Fast prototyping needed

How it works:

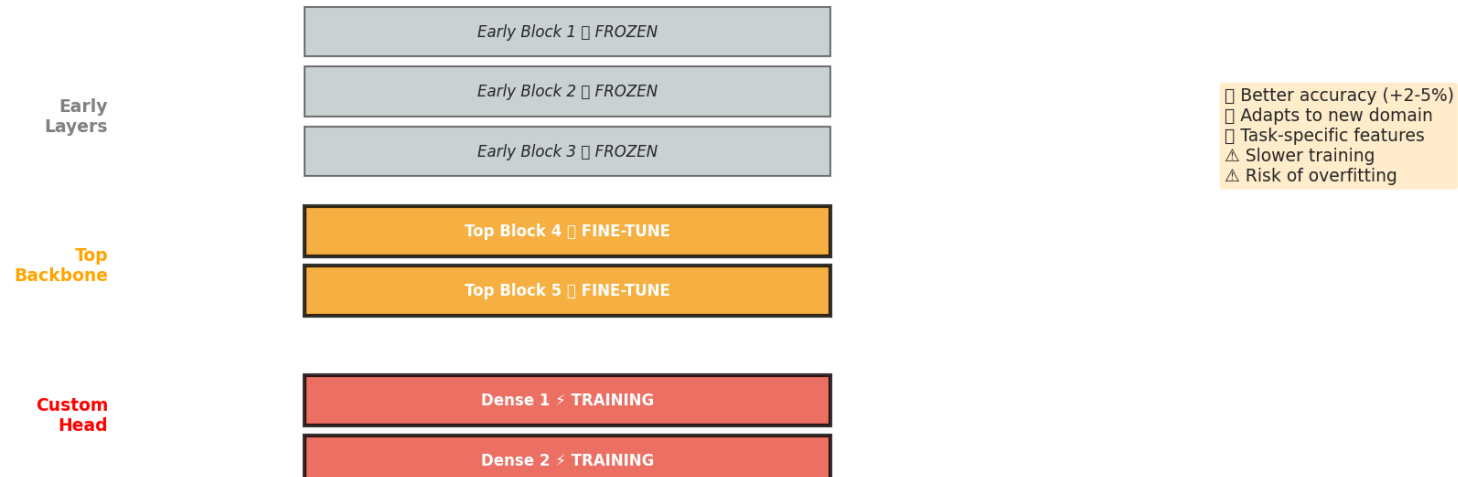
1. Load pretrained model (ResNet50 from ImageNet)
2. **Freeze all backbone layers** (no training)
3. Remove original head (1000 ImageNet classes)
4. Add new head (3 crop disease classes)
5. Train only the new head

Fine-Tuning Workflow

Fine-Tuning Workflow: Adapting Pretrained Features

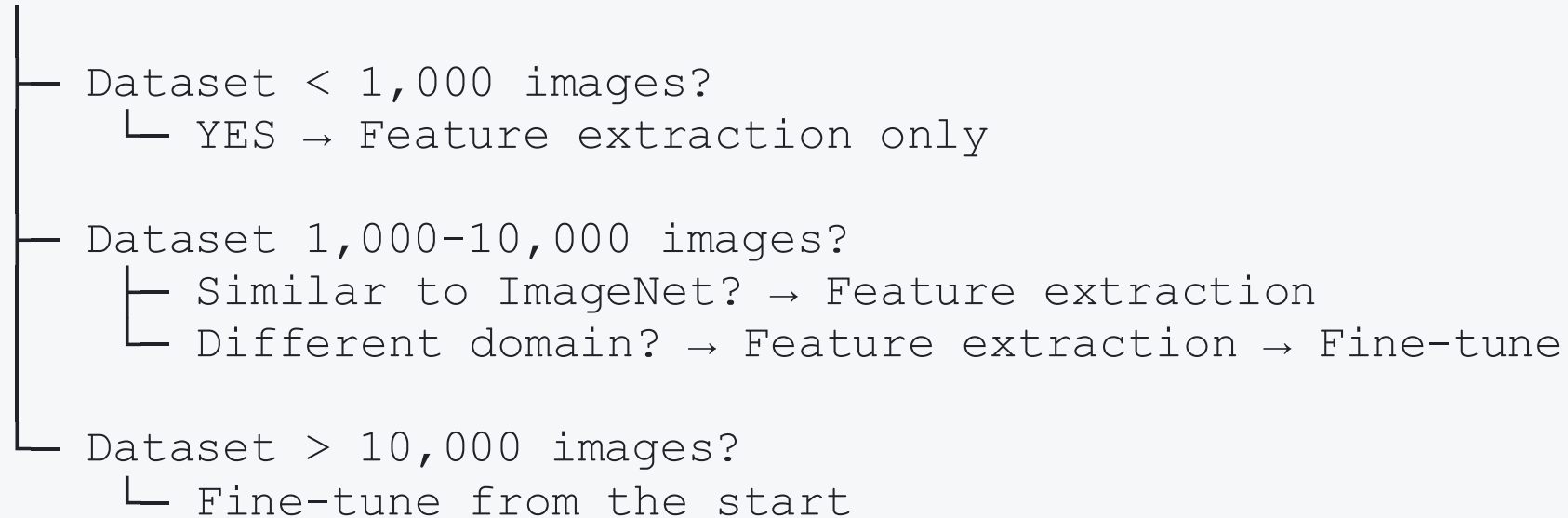


Detailed View:



When to Freeze vs Fine-Tune: Decision Tree

Start here



Nigerian crop disease (~1,500 images):

Start with feature extraction → Try fine-tuning if needed

Why Reduce Learning Rate for Fine-Tuning?

Pretrained weights are already GOOD.

Large LR ($1e-3$):

- Big weight updates
- Can destroy learned features
- Loss explodes



Small LR ($1e-5$):

- Gentle weight adjustments
- Adapts features without breaking them
- Stable improvement

Analogy: Tuning a piano (gentle adjustments) vs rebuilding it (destructive)



Part 3: Data Augmentation

Why Augmentation Matters

Problem: Neural networks overfit when data is limited

Solution: Artificially increase dataset size by creating variations

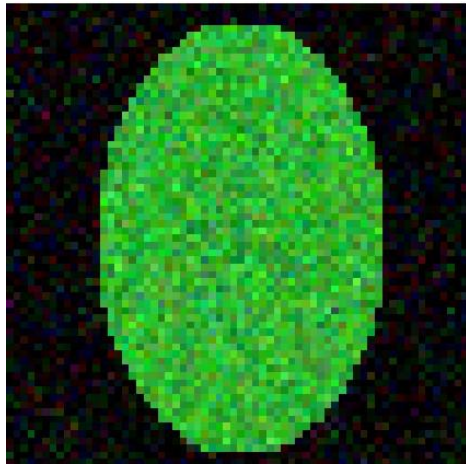
Nigerian crop disease example:

- Original: 500 cassava leaf images
- After augmentation: **Infinite** variations!
- Each training epoch sees slightly different images

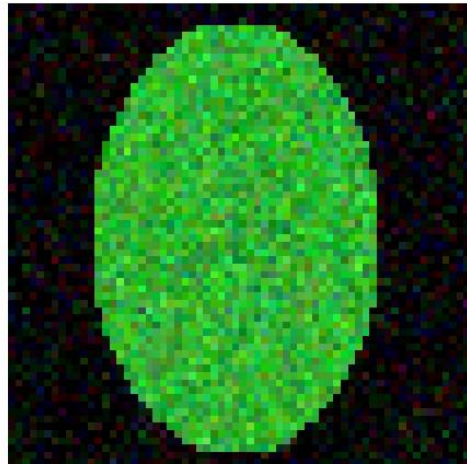
Augmentation Examples

Data Augmentation Examples: Nigerian Crop Disease Detection

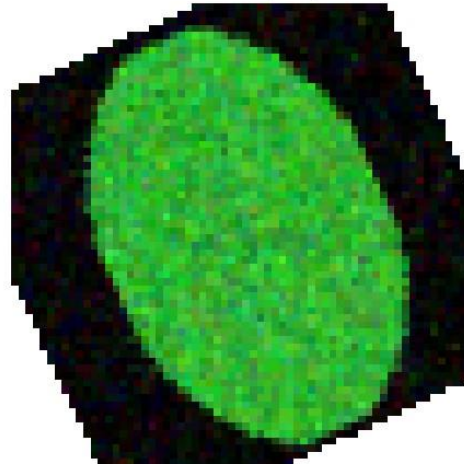
Original
(Cassava Leaf)



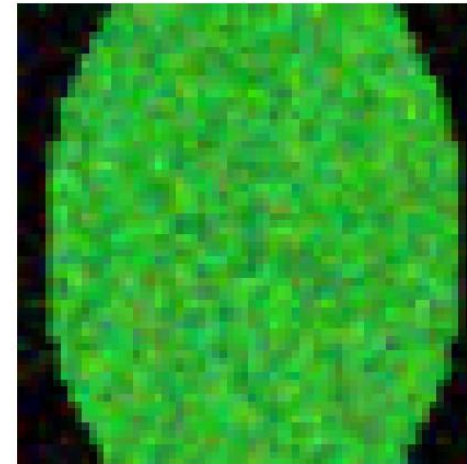
Horizontal Flip



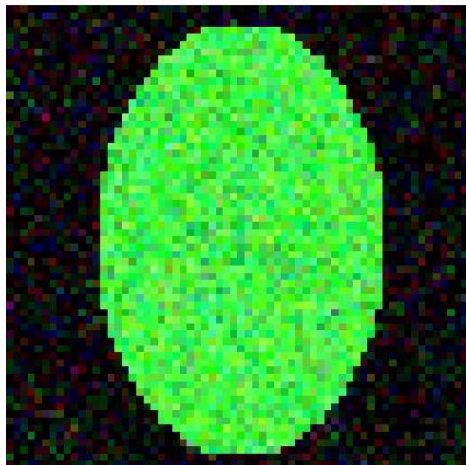
Rotation (25°)



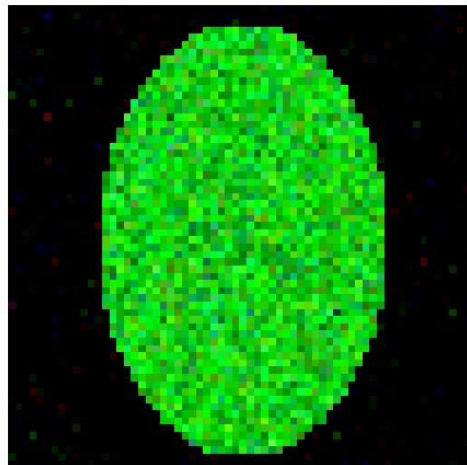
Zoom/Crop



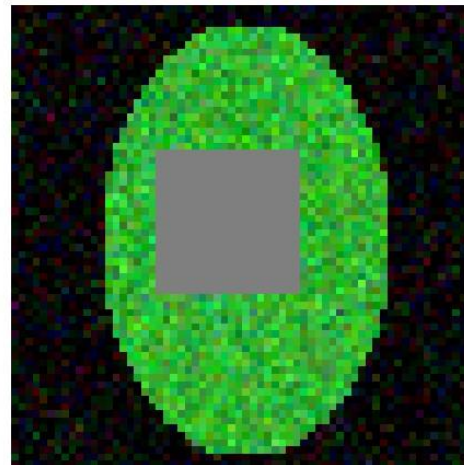
Brightness ↑



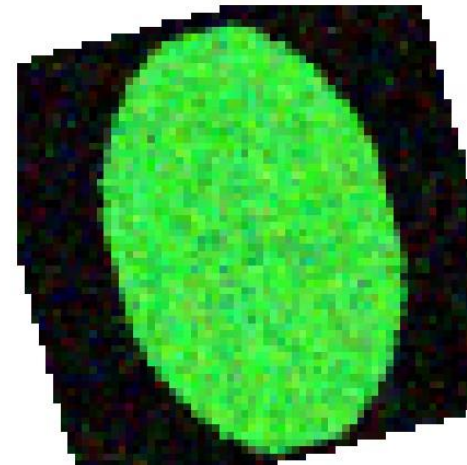
Contrast ↑



Random Cutout



Combined
(Rotation+Brightness)



Augmentation Techniques:

The Essential Toolkit

Technique	What it does	When to use
Horizontal Flip	Mirror image	Always (unless asymmetry matters)
Rotation	Rotate $\pm 10\text{-}30^\circ$	Objects can appear at angles
Zoom/Crop	Random zoom in/out	Vary object size in frame
Brightness	Adjust exposure	Lighting variations (farms!)
Contrast	Adjust contrast	Camera quality variations
Cutout	Random patches removed	Occlusion robustness

For Nigerian crop disease: Use ALL of these! Farm conditions vary wildly.

Augmentation in Keras/TensorFlow

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,           # ±30° rotation
    width_shift_range=0.2,       # 20% horizontal shift
    height_shift_range=0.2,      # 20% vertical shift
    shear_range=0.2,             # Shear transformation
    zoom_range=0.2,              # 20% zoom in/out
    horizontal_flip=True,        # Random horizontal flip
    brightness_range=[0.8, 1.2], # ±20% brightness
    fill_mode='nearest'         # Fill missing pixels
)

train_generator = train_datagen.flow_from_directory(
    'data/train',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
```

Augmentation Best Practices

Do:

-  Apply augmentation to **training set only** (not validation/test)
-  Use realistic transformations (farms have rotation, lighting changes)
-  Start conservative, increase gradually
-  Visualize augmented images to verify they look natural

Don't:

-  Over-augment (e.g., 180° rotation for leaves makes no sense)
-  Augment validation/test sets (cheating!)
-  Use augmentations that change semantics (e.g., vertical flip for text)

Advanced Augmentation (Bonus)

MixUp: Blend two images and their labels

```
# Image A (cassava mosaic) + Image B (healthy) = Mixed image  
# Label: 0.7 * mosaic + 0.3 * healthy
```

CutMix: Cut patch from one image, paste into another

- More realistic than MixUp
- Forces model to look at multiple regions

For assignment bonus: Implement MixUp and compare!

Part 4: Optimization Techniques

Early Stopping: Preventing Overfitting

Problem:

- Training loss keeps decreasing ↓
- Validation loss starts increasing ↑
- Model is memorizing training data!

Solution: Early stopping

- Monitor validation loss
- Stop when it stops improving
- Restore best weights

Early Stopping in Keras

```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',           # Watch validation loss
    patience=5,                   # Wait 5 epochs for improvement
    restore_best_weights=True,    # Revert to best model
    verbose=1
)

model.fit(
    train_generator,
    epochs=50,
    validation_data=val_generator,
    callbacks=[early_stop]        # Add callback
)
```

Result: Training stops at epoch 28 instead of 50 → saves time + prevents overfitting!

Learning Rate Scheduling

Problem: Fixed LR is suboptimal

- Early training: Need large LR to explore
- Late training: Need small LR to fine-tune

Solution: ReduceLROnPlateau

- Reduce LR when validation loss plateaus
- Automatic fine-tuning!

LR Scheduler in Keras

```
from tensorflow.keras.callbacks import ReduceLROnPlateau

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,           # Reduce LR by half
    patience=3,           # Wait 3 epochs
    min_lr=1e-7,          # Don't go below this
    verbose=1
)

model.fit(
    train_generator,
    epochs=50,
    callbacks=[early_stop, reduce_lr]  # Combine callbacks
)
```

Example: LR $1e-3 \rightarrow 5e-4 \rightarrow 2.5e-4 \rightarrow \dots$ automatically!

Model Checkpointing

Save best model during training:

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint(
    'best_model.h5',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

model.fit(
    train_generator,
    epochs=50,
    callbacks=[early_stop, reduce_lr, checkpoint]
)
```

Benefit: Never lose your best model if training crashes!

TensorBoard: Visualizing Training

Real-time monitoring:

```
from tensorflow.keras.callbacks import TensorBoard

tensorboard = TensorBoard(
    log_dir='./logs',
    histogram_freq=1
)

model.fit(
    train_generator,
    callbacks=[early_stop, reduce_lr, checkpoint, tensorboard
```

View in browser:

```
tensorboard --logdir=./logs
```

Part 5: Practical Implementation

Complete Transfer Learning Pipeline

Step 1: Prepare data (split into train/val/test)

Step 2: Set up augmentation (train only)

Step 3: Load pretrained model (ResNet50 or EfficientNetB0)

Step 4: Feature extraction (freeze backbone, train head, 10-20 epochs)

Step 5: Evaluate baseline performance

Step 6: Fine-tuning (unfreeze top layers, small LR, 5-10 epochs)

Step 7: Final evaluation + confusion matrix

Step 8: Deploy to mobile app (TensorFlow Lite conversion)

Backbone Comparison:

ResNet50 vs EfficientNetB0

For Nigerian crop disease classification:

Metric	ResNet50	EfficientNetB0
Parameters	25M	5.3M
ImageNet Accuracy	76%	77%
Speed (inference)	45ms	30ms
Mobile-friendly?	Medium	Yes 
Memory usage	100MB	20MB

Recommendation: Start with ResNet50 (familiar), then try EfficientNetB0 for deployment

When Transfer Learning Fails

Scenarios where transfer learning doesn't help much:

1. Extremely different domains:

- ImageNet (natural images) → Medical X-rays (better to pretrain on ChestX-ray14)
- ImageNet → Satellite imagery (better to pretrain on fMoW)

2. Very large datasets:

- 1M+ labeled images → Training from scratch might match transfer learning

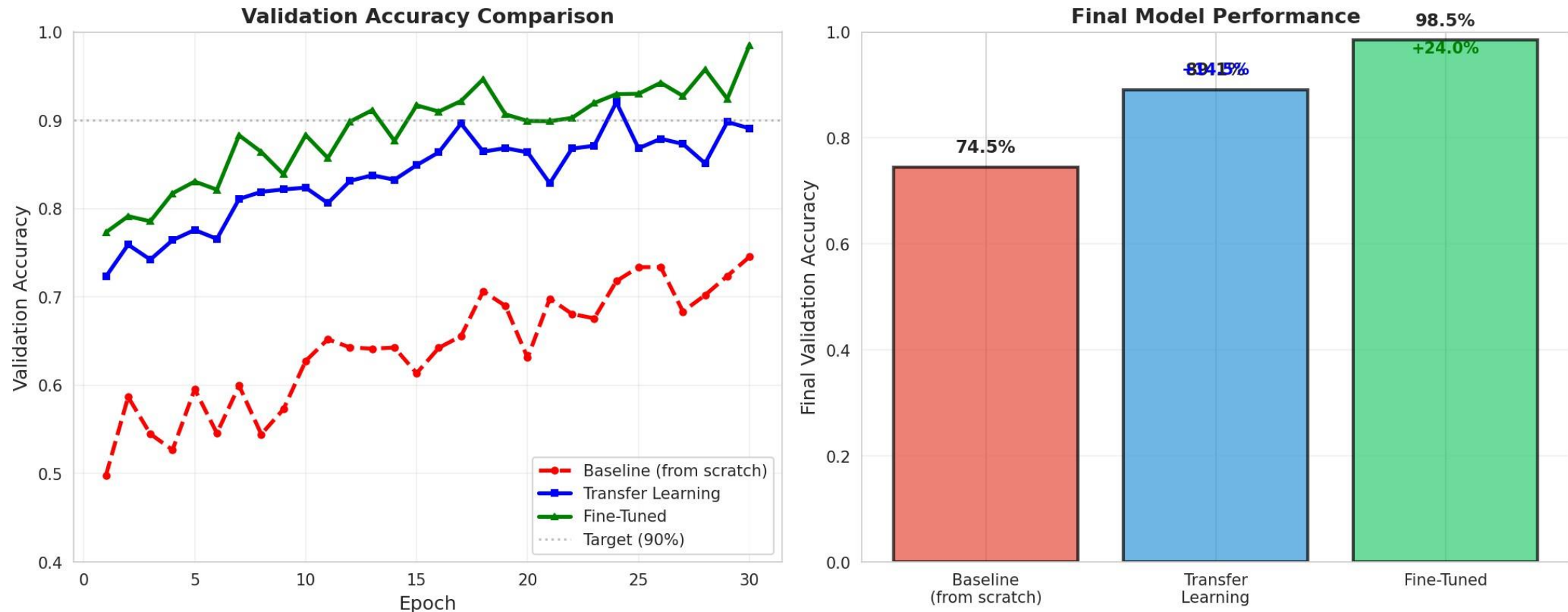
3. Tiny images:

- ImageNet is 224×224 , but your data is 32×32 (CIFAR-10) → Resolution mismatch

Part 6: Comparison & Results

Learning Curves: The Proof

Transfer Learning Dramatically Improves Performance: Nigerian Crop Disease Detection



Transfer learning reaches 88% in 10 epochs. Baseline needs 30 epochs for 75%!

Performance Metrics Summary

Nigerian crop disease classification results:

Approach	Val Accuracy	Training Time	GPU Hours
From scratch	75%	6 hours	0.5
Feature extraction	88%	15 minutes	0.05
Fine-tuned	92%	30 minutes	0.1

Cost savings: Transfer learning saves 90% of GPU time + 10%+ accuracy!

Confusion Matrix Analysis

Fine-tuned ResNet50 on Nigerian crop disease:

	Pred: Healthy	Pred: Mosaic	Pred: Blight
True: Healthy	95%	3%	2%
True: Mosaic	5%	90%	5%
True: Blight	2%	4%	94%

Analysis:

- High precision on all classes 
- Occasional confusion between Mosaic and Blight (similar symptoms)
- Healthy leaves rarely misclassified

Business Impact

Mobile app for Nigerian farmers:

Before (manual extension officers):

- Diagnosis time: 2-7 days
- Cost per diagnosis: ₦5,000
- Coverage: Limited to major farms

After (CNN mobile app):

- Diagnosis time: **Instant** (< 1 second)
- Cost per diagnosis: **₦0** (free app)
- Coverage: **Any farmer with smartphone**

Impact:

Summary & Key Takeaways

What We Learned Today

1.  **Modern CNN architectures:** ResNet (skip connections), EfficientNet (compound scaling)
2.  **Transfer learning modes:** Feature extraction (freeze) vs fine-tuning (unfreeze)
3.  **Data augmentation:** Flip, rotate, zoom, brightness → infinite variations
4.  **Optimization:** Early stopping, LR scheduling, model checkpointing
5.  **Practical workflow:** Load pretrained → Freeze → Train head → Fine-tune → Deploy
6.  **Nigerian crop disease:** 92% accuracy with transfer learning vs 75% from scratch

The Big Picture: Why Transfer Learning Works

Universal feature hierarchy:

- **Layer 1-2:** Edges, corners, colors (useful for EVERYTHING)
- **Layer 3-5:** Textures, simple shapes (dogs, digits, leaves all have these)
- **Layer 6-10:** Object parts (eyes, ears, petals)
- **Layer 11+:** Task-specific (dog breeds vs crop diseases)

Transfer learning:

- Reuse universal features (layers 1-10)
- Retrain task-specific features (layers 11+)

Result: 90% less data, 10× faster training, better accuracy!

Best Practices Checklist

Do:

- Start with feature extraction, then try fine-tuning
- Use data augmentation (especially for small datasets)
- Reduce LR when fine-tuning ($1e-5$ instead of $1e-3$)
- Monitor with early stopping + LR scheduling
- Visualize augmented samples to verify naturalness
- Compare multiple backbones (ResNet50, EfficientNetB0)

Don't:

- Train large models from scratch (waste of time/money)
 - Augment validation/test sets
- Use large LR for fine-tuning (destroys pretrained weights)

Practice Exercises & Assignment

Practice Exercises (8 exercises):

- Load ResNet50 with ImageNet weights
- Feature extraction on Nigerian crop disease
- Implement data augmentation pipeline
- Fine-tune top layers
- Compare ResNet50 vs EfficientNetB0
- Learning rate experiments
- Visualize learned features
- Deploy with TensorFlow Lite

Assignment (100 points):

- Full transfer learning pipeline

Next Week: NLP Fundamentals

Moving from images to text:

- Word embeddings (Word2Vec, GloVe)
- Recurrent Neural Networks (RNNs, LSTMs)
- Text classification (Nigerian news categorization)
- Sentiment analysis (product reviews)

Get excited! We're expanding beyond computer vision 